

✓ 08 — Model evaluation and comparison

I combine representation geometry and decoding results into benchmark tables. This notebook is deliberately comparative: CEBRA is not treated as successful unless it improves meaningful metrics over baselines.

```
from pathlib import Path
import sys

ROOT = Path.cwd()
if ROOT.name == "notebooks":
    ROOT = ROOT.parent
if str(ROOT / "src") not in sys.path:
    sys.path.insert(0, str(ROOT / "src"))

from v1_manifold.config import load_config, get_paths, set_global_seed
from v1_manifold.visualization import set_publication_style, save_figure
from v1_manifold.utils import save_table

cfg = load_config(ROOT / "configs" / "default.yaml")
cfg["paths"]["root"] = str(ROOT)
paths = get_paths(cfg)
set_global_seed(cfg["project"]["random_seed"])
set_publication_style()
print(f"Project root: {ROOT}")
```

Project root: c:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from v1_manifold.models import valid_embedding_names, describe_embedding_file
from v1_manifold.visualization import plot_metric_bar

# -----
# Robust geometry-table loading or reconstruction
# -----
def trajectory_geometry_metrics(Z):
    """Compute lightweight geometry metrics from a frame-level latent trajectory
    Z = np.asarray(Z, dtype=float)

    if Z.ndim == 1:
        Z = Z.reshape(-1, 1)
```

```

diffs = np.diff(Z, axis=0)
step_lengths = np.linalg.norm(diffs, axis=1)

# Participation ratio of latent covariance spectrum.
Zc = Z - Z.mean(axis=0, keepdims=True)
cov = np.cov(Zc, rowvar=False)

if np.ndim(cov) == 0:
    eigvals = np.array([float(cov)])
else:
    eigvals = np.linalg.eigvalsh(cov)

eigvals = np.clip(eigvals, 0, None)
participation_ratio = float((eigvals.sum() ** 2) / (np.sum(eigvals ** 2))

# Simple discrete curvature proxy from turning angle per arc length.
if len(Z) >= 3:
    v1 = Z[1:-1] - Z[:-2]
    v2 = Z[2:] - Z[1:-1]

    n1 = np.linalg.norm(v1, axis=1)
    n2 = np.linalg.norm(v2, axis=1)

    valid = (n1 > 1e-12) & (n2 > 1e-12)

    curvature = np.full(len(v1), np.nan)

    if valid.any():
        cosang = np.sum(v1[valid] * v2[valid], axis=1) / (n1[valid] * n2[valid])
        cosang = np.clip(cosang, -1, 1)
        angles = np.arccos(cosang)
        local_arc = 0.5 * (n1[valid] + n2[valid])
        curvature[valid] = angles / (local_arc + 1e-12)

    mean_curvature = float(np.nanmean(curvature))
    median_curvature = float(np.nanmedian(curvature))
else:
    mean_curvature = np.nan
    median_curvature = np.nan

return {
    "n_frames": int(Z.shape[0]),
    "embedding_dim": int(Z.shape[1]),
    "participation_ratio": participation_ratio,
    "trajectory_length": float(np.sum(step_lengths)),
    "mean_step_length": float(np.mean(step_lengths)),
    "median_step_length": float(np.median(step_lengths)),
    "mean_discrete_curvature": mean_curvature,
    "median_discrete_curvature": median_curvature,
}

```

```

def load_or_create_geometry_summary(paths):
    expected_path = paths.tables_dir / "05_geometry_summary.csv"

    if expected_path.exists():
        print(f"Loaded geometry summary: {expected_path}")
        return pd.read_csv(expected_path)

    candidate_paths = sorted(paths.tables_dir.glob("05*geometry*.csv"))

    for candidate in candidate_paths:
        candidate_df = pd.read_csv(candidate)

        if {"session_id", "embedding"}.issubset(candidate_df.columns):
            print(f"Loaded geometry summary from alternate file: {candidate}")
            save_table(candidate_df, expected_path)
            return candidate_df

    print("No existing geometry summary found. Reconstructing geometry summary")

    emb_files = sorted(paths.processed_dir.glob("session*_embeddings.npz"))

    if not emb_files:
        raise FileNotFoundError(
            "No session embedding file found in data/processed/. "
            "I need notebook 05 or 07 to have saved session_<id>_embeddings.npz"
        )

    rows = []

    for emb_path in emb_files:
        session_id_local = emb_path.name.split("_")[1]

        with np.load(emb_path, allow_pickle=False) as emb_file:
            names = valid_embedding_names(emb_file, n_samples=None)

            # Fallback if valid_embedding_names requires n_samples.
            if not names:
                possible_n = None
                if "frame" in emb_file.files:
                    possible_n = len(emb_file["frame"])

                if possible_n is not None:
                    names = valid_embedding_names(emb_file, n_samples=possible_n)

            for embedding_name in names:
                Z = np.asarray(emb_file[embedding_name])

                if Z.ndim != 2:
                    continue

```

```

        metrics = trajectory_geometry_metrics(Z)

        rows.append({
            "session_id": int(session_id_local),
            "embedding": embedding_name,
            **metrics,
            "geometry_source": "reconstructed_from_embedding_file",
        })

    geometry = pd.DataFrame(rows)

    if geometry.empty:
        raise RuntimeError("Could not reconstruct geometry summary from saved")

    save_table(geometry, expected_path)
    print(f"Saved reconstructed geometry summary: {expected_path}")

    return geometry

geometry = load_or_create_geometry_summary(paths)

display(geometry)

```

```

No existing geometry summary found. Reconstructing geometry summary from save
Saved reconstructed geometry summary: C:\Users\Peter\Documents\projects\Neuro

```

	session_id	embedding	n_frames	embedding_dim	participation_ratio	traje
0	500855614	pca	900	3	2.790296	
1	500855614	pca_full	900	20	16.302486	
2	500855614	umap	900	3	2.154747	
3	500855614	isomap	900	3	1.990444	
4	500855614	cebra	900	3	2.999122	

```

# -----
# Helper functions
# -----

def load_csv_or_empty(path):
    path = Path(path)
    if path.exists():
        return pd.read_csv(path)
    return pd.DataFrame()

def load_matching_csvs(pattern):
    files = sorted(paths.tables_dir.glob(pattern))

```

```

    if not files:
        return pd.DataFrame()

    tables = []

    for file in files:
        df = pd.read_csv(file)
        df["source_file"] = file.name
        tables.append(df)

    return pd.concat(tables, ignore_index=True)

def normalize_session_id(df):
    if not df.empty and "session_id" in df.columns:
        df = df.copy()
        df["session_id"] = df["session_id"].astype(str)
    return df

def robust_false_mask(series):
    """
    Return True for rows that are not null/permutation baselines.
    This is safer than astype(bool), because bool("False") is True in Python
    """
    return ~series.astype(str).str.lower().isin(["true", "1", "yes"])

# -----
# Make sure geometry exists in this notebook
# -----
if "geometry" not in globals():
    geometry_path = paths.tables_dir / "05_geometry_summary.csv"

    if not geometry_path.exists():
        raise FileNotFoundError(
            "Geometry summary is missing. Run the previous geometry reconstr
        )

    geometry = pd.read_csv(geometry_path)

geometry = normalize_session_id(geometry)

# -----
# Load notebook 06 and notebook 07 outputs
# -----
decoding_path = paths.tables_dir / "06_baseline_decoding_benchmark.csv"
regression_path = paths.tables_dir / "06_continuous_feature_decoding_benchma

```

```

if not decoding_path.exists():
    raise FileNotFoundError("Decoding benchmark missing. Run notebook 06 or

decoding = pd.read_csv(decoding_path)
regression = load_csv_or_empty(regression_path)

# I load all session-specific CEBRA and dRNN outputs without relying on a se
cebra_comparison = load_matching_csvs(
    "07_cebra_vs_other_embeddings_feature_regression_session_*.csv"
)

cebra_summary = load_matching_csvs(
    "07_cebra_representation_summary_session_*.csv"
)

drnn_baseline = load_matching_csvs(
    "07_drnn_next_state_baseline_comparison_session_*.csv"
)

drnn_history = load_matching_csvs(
    "07_drnn_training_history_session_*.csv"
)

drnn_metrics = load_matching_csvs(
    "07_drnn_next_state_metrics_session_*.csv"
)

decoding = normalize_session_id(decoding)
regression = normalize_session_id(regression)
cebra_comparison = normalize_session_id(cebra_comparison)
cebra_summary = normalize_session_id(cebra_summary)
drnn_baseline = normalize_session_id(drnn_baseline)
drnn_history = normalize_session_id(drnn_history)
drnn_metrics = normalize_session_id(drnn_metrics)

# -----
# Filter null baselines before selecting best decoders
# -----
decoding_non_null = decoding.copy()

if "is_null_baseline" in decoding_non_null.columns:
    decoding_non_null = decoding_non_null[
        robust_false_mask(decoding_non_null["is_null_baseline"])
    ].copy()

required_decoding_cols = {"session_id", "embedding", "target", "analysis", "
missing_decoding_cols = required_decoding_cols - set(decoding_non_null.colum
if missing_decoding_cols:

```

```

        raise KeyError(f"Decoding table is missing required columns: {missing_de

best_decoding = (
    decoding_non_null
    .sort_values("balanced_accuracy", ascending=False)
    .groupby(["session_id", "embedding", "target", "analysis"], as_index=False)
    .first()
)

# -----
# Integrate geometry with best decoding results
# -----
integrated = geometry.merge(
    best_decoding,
    on=["session_id", "embedding"],
    how="left",
)

save_table(
    integrated,
    paths.tables_dir / "08_integrated_benchmark_table.csv",
)

if not regression.empty:
    save_table(
        regression,
        paths.tables_dir / "08_continuous_feature_benchmark_table.csv",
    )

if not ceбра_comparision.empty:
    save_table(
        ceбра_comparision,
        paths.tables_dir / "08_ceбра_embedding_feature_benchmark_table.csv",
    )

if not ceбра_summary.empty:
    save_table(
        ceбра_summary,
        paths.tables_dir / "08_ceбра_representation_summary_table.csv",
    )

if not drnn_baseline.empty:
    save_table(
        drnn_baseline,
        paths.tables_dir / "08_drnn_next_state_baseline_table.csv",
    )

if not drnn_history.empty:
    save_table(
        drnn_history,
        paths.tables_dir / "08_drnn_history_table.csv",
    )

```

```

        drnn_nhistory,
        paths.tables_dir / "08_drnn_training_history_table.csv",
    )

if not drnn_metrics.empty:
    save_table(
        drnn_metrics,
        paths.tables_dir / "08_drnn_next_state_metrics_table.csv",
    )

# -----
# Display compact status summary
# -----
loaded_status = pd.DataFrame([
    {"table": "geometry", "rows": len(geometry), "loaded": not geometry.empty},
    {"table": "decoding", "rows": len(decoding), "loaded": not decoding.empty},
    {"table": "regression", "rows": len(regression), "loaded": not regression.empty},
    {"table": "cebra_comparison", "rows": len(cebra_comparison), "loaded": not cebra_comparison.empty},
    {"table": "cebra_summary", "rows": len(cebra_summary), "loaded": not cebra_summary.empty},
    {"table": "drnn_baseline", "rows": len(drnn_baseline), "loaded": not drnn_baseline.empty},
    {"table": "drnn_history", "rows": len(drnn_history), "loaded": not drnn_history.empty},
    {"table": "drnn_metrics", "rows": len(drnn_metrics), "loaded": not drnn_metrics.empty}
])

save_table(
    loaded_status,
    paths.tables_dir / "08_loaded_results_status.csv",
)

display(loaded_status)
display(integrated.head())

```

	table	rows	loaded				
0	geometry	5	True				
1	decoding	160	True				
2	regression	20	True				
3	cebra_comparison	25	True				
4	cebra_summary	5	True				
5	drnn_baseline	3	True				
6	drnn_history	20	True				
7	drnn_metrics	1	True				
	session_id	embedding	n_frames	embedding_dim	participation_ratio	traje	
0	500855614	pca	900	3	2.790296		

1	500855614	pca	900	3	2.790296
2	500855614	pca	900	3	2.790296
3	500855614	pca	900	3	2.790296
4	500855614	pca_full	900	20	16.302486

✓ Geometry summary plots

```
# -----
# Geometry summary across embeddings
# -----
geometry_summary = (
    geometry
    .sort_values("participation_ratio", ascending=False)
    .reset_index(drop=True)
)

save_table(
    geometry_summary,
    paths.tables_dir / "08_geometry_summary_by_embedding.csv",
)

display(geometry_summary)

fig, ax = plt.subplots(figsize=(7, 4))
ax.bar(geometry_summary["embedding"], geometry_summary["participation_ratio"])
ax.set_xlabel("Embedding")
ax.set_ylabel("Participation ratio")
ax.set_title("Latent dimensionality by representation")
ax.tick_params(axis="x", rotation=25)

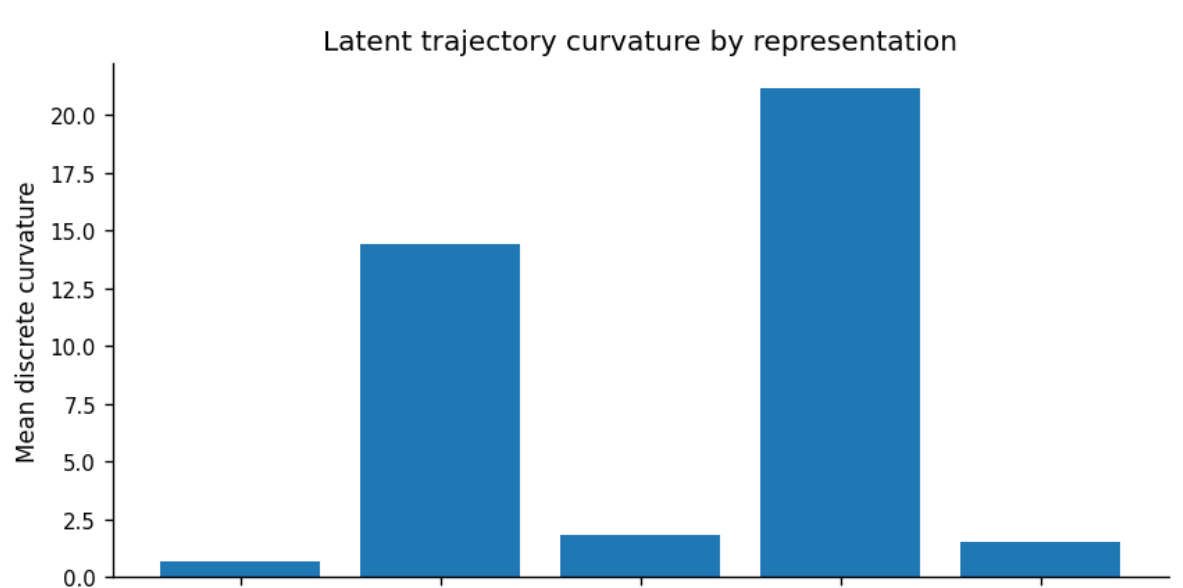
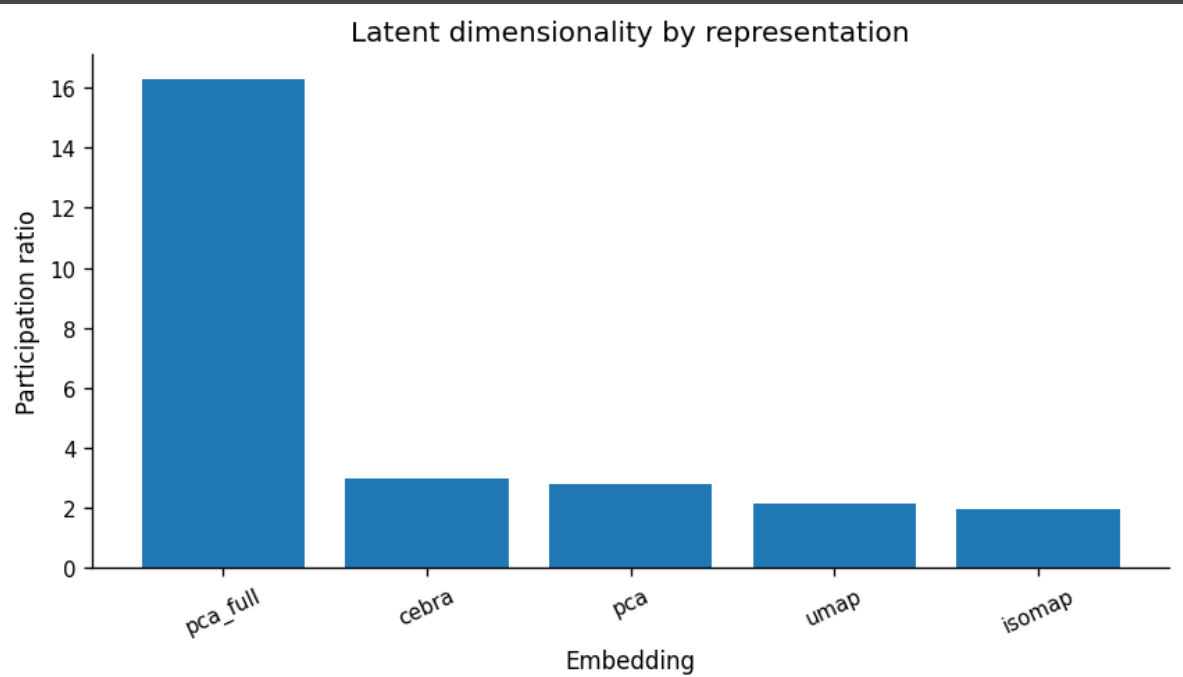
save_figure(
    fig,
    paths.figures_dir / "08_geometry_participation_ratio_by_embedding.png",
)

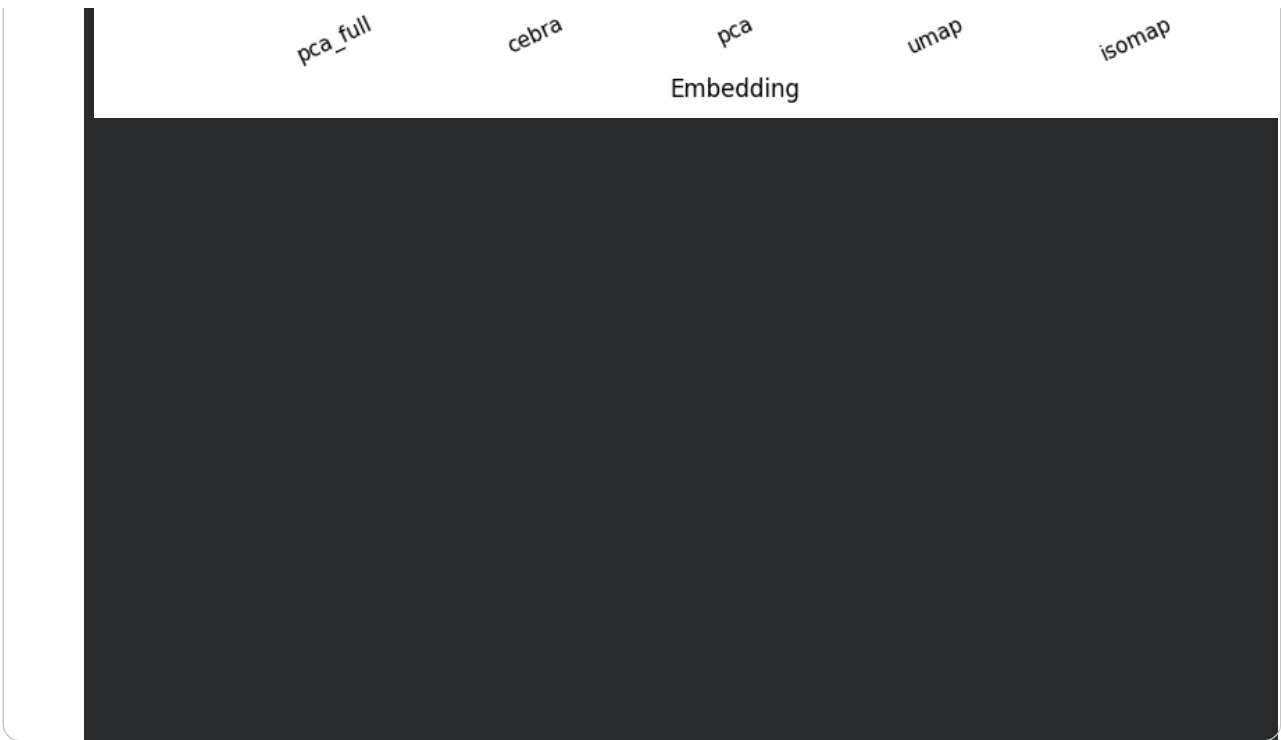
plt.show()

fig, ax = plt.subplots(figsize=(7, 4))
ax.bar(geometry_summary["embedding"], geometry_summary["mean_discrete_curvat"])
ax.set_xlabel("Embedding")
ax.set_ylabel("Mean discrete curvature")
ax.set_title("Latent trajectory curvature by representation")
ax.tick_params(axis="x", rotation=25)
```

```
save_figure(  
    fig,  
    paths.figures_dir / "08_geometry_curvature_by_embedding.png",  
)  
  
plt.show()
```

	session_id	embedding	n_frames	embedding_dim	participation_ratio	traje
0	500855614	pca_full	900	20	16.302486	
1	500855614	cebra	900	3	2.999122	
2	500855614	pca	900	3	2.790296	
3	500855614	umap	900	3	2.154747	
4	500855614	isomap	900	3	1.990444	





✓ Best classification decoding results

```
# -----  
# Best non-null classification decoders  
# -----  
classification_results = integrated.dropna(subset=["balanced_accuracy"]).copy()  
  
best_classification = (  
    classification_results  
    .sort_values("balanced_accuracy", ascending=False)  
    .groupby(["target", "analysis"], as_index=False)  
    .first()  
    .sort_values("balanced_accuracy", ascending=False)  
)  
  
save_table(  
    best_classification,  
    paths.tables_dir / "08_best_classification_decoders.csv",  
)  
  
display(best_classification[  
    [  
        "session_id",  
        "target",  
        "analysis",  
        "embedding",  
        "model",  
        "accuracy",  
        "balanced accuracy".
```

```

        "macro_f1",
        "n_samples",
        "n_features",
    ]
].head(20))

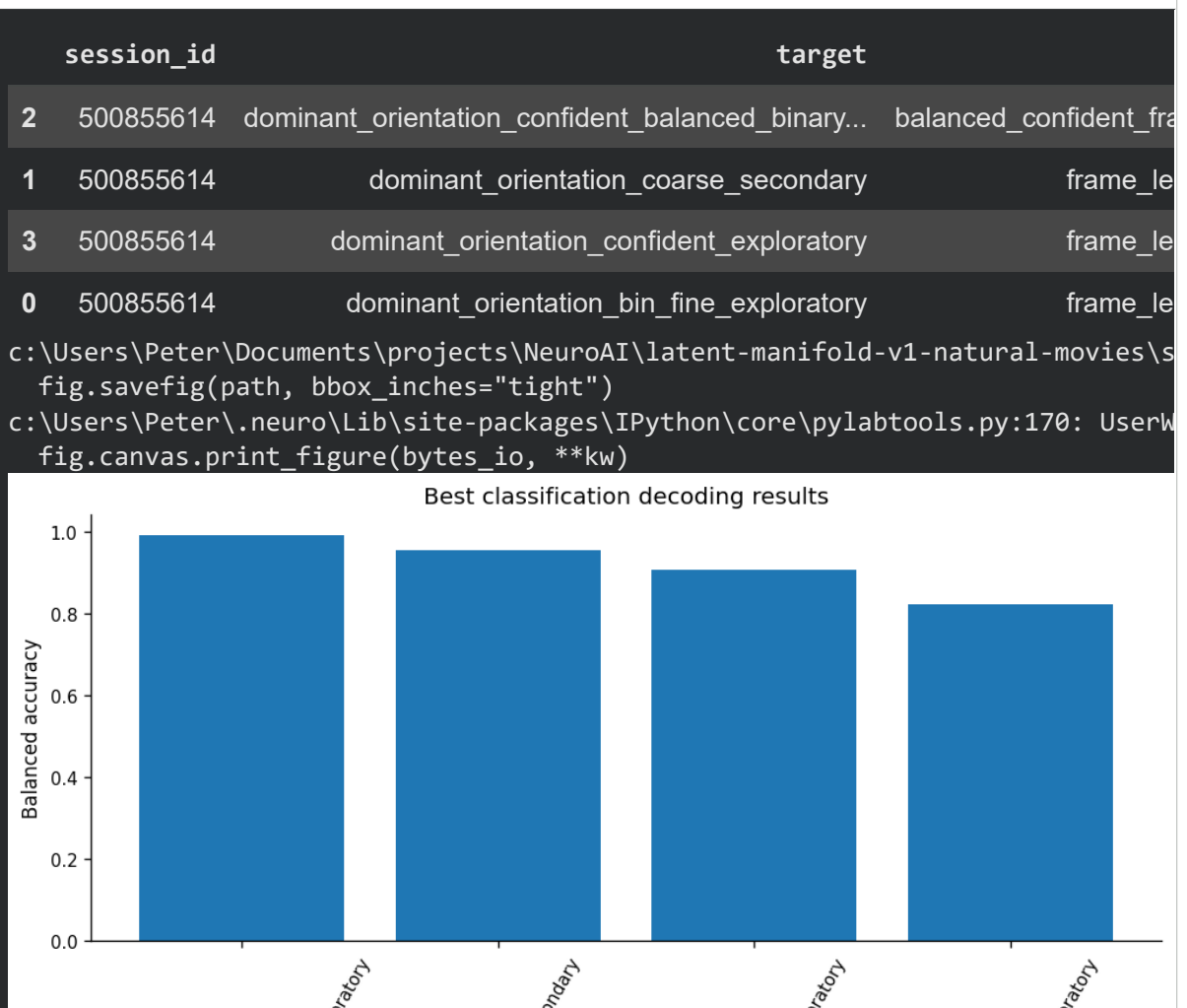
plot_df = best_classification.head(12).copy()
plot_df["label"] = plot_df["target"] + "\n" + plot_df["embedding"].astype(st

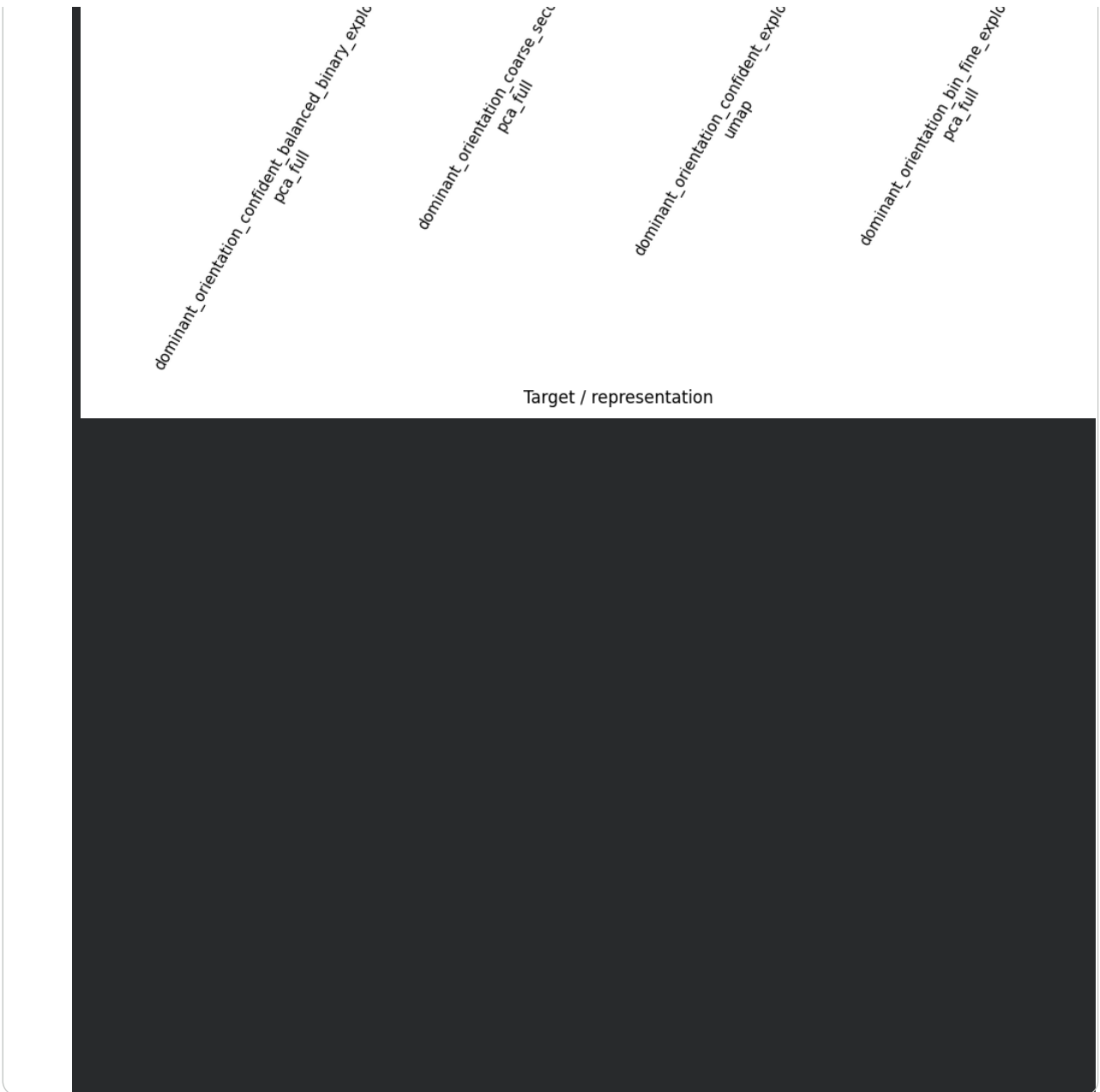
fig, ax = plt.subplots(figsize=(10, 4))
ax.bar(plot_df["label"], plot_df["balanced_accuracy"])
ax.set_xlabel("Target / representation")
ax.set_ylabel("Balanced accuracy")
ax.set_title("Best classification decoding results")
ax.tick_params(axis="x", rotation=60)

save_figure(
    fig,
    paths.figures_dir / "08_best_classification_decoding_results.png",
)

plt.show()

```





✓ Continuous real-feature decoding summary

```
# -----  
# Continuous real movie-feature decoding summary  
# -----  
if not regression.empty:  
    regression_best = (  
        regression  
        .sort_values("r2", ascending=False)  
        .groupby("target", as_index=False)  
        .first()  
        .sort_values("r2", ascending=False)  
    )  
  
save_table/
```

```

save_table(
    regression_best,
    paths.tables_dir / "08_best_continuous_feature_decoders.csv",
)

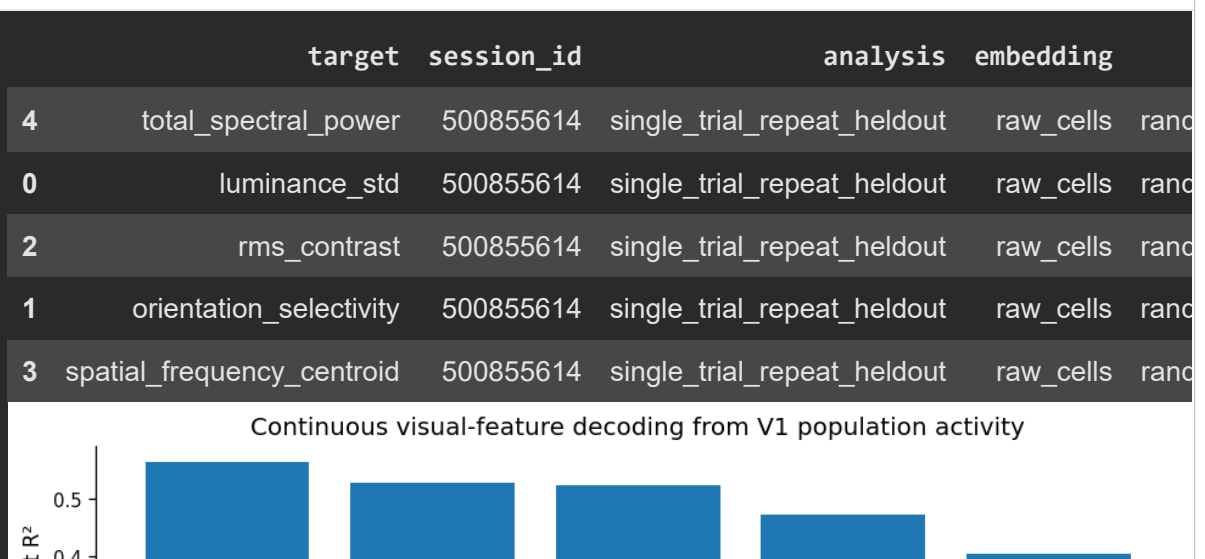
display(regression_best[
    [
        "target",
        "session_id",
        "analysis",
        "embedding",
        "model",
        "mae",
        "rmse",
        "r2",
        "n_samples",
        "n_features",
    ]
])

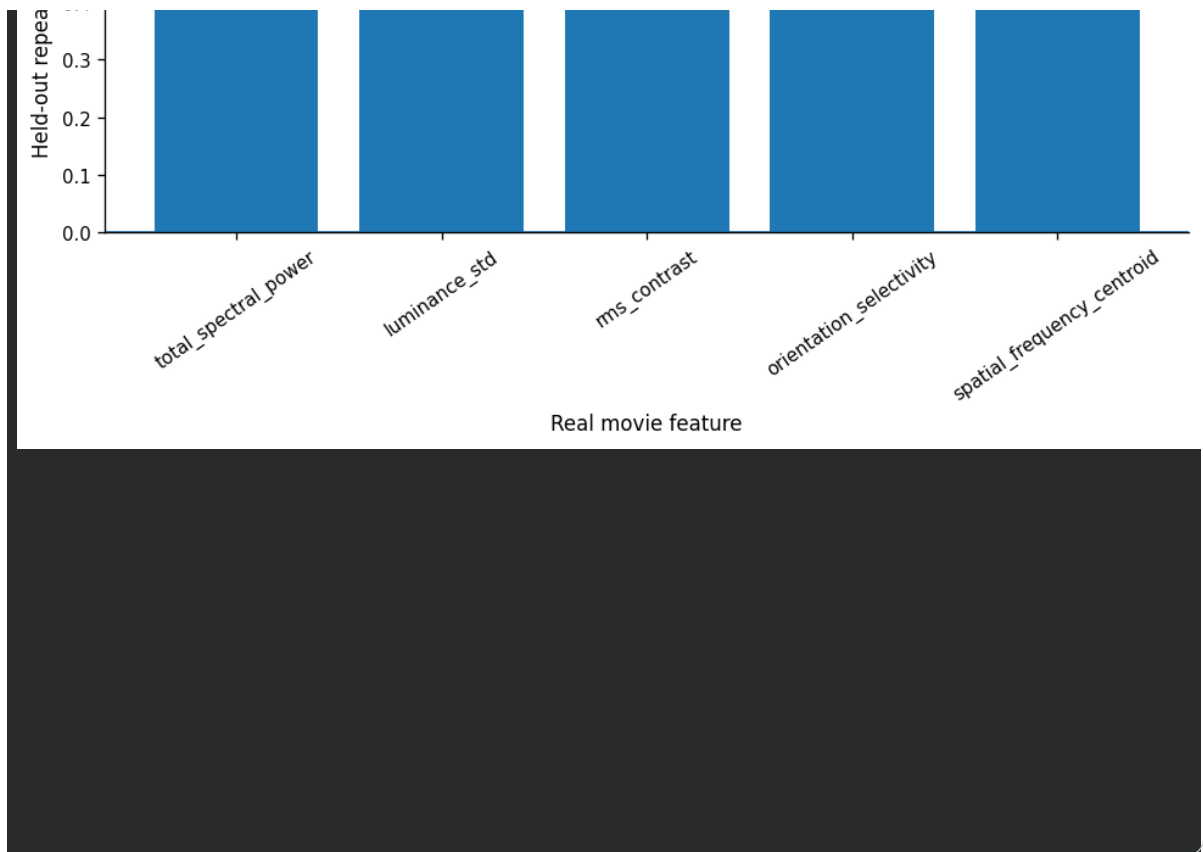
fig, ax = plt.subplots(figsize=(8, 4))
ax.bar(regression_best["target"], regression_best["r2"])
ax.axhline(0, linewidth=1)
ax.set_xlabel("Real movie feature")
ax.set_ylabel("Held-out repeat R2")
ax.set_title("Continuous visual-feature decoding from V1 population acti
ax.tick_params(axis="x", rotation=35)

save_figure(
    fig,
    paths.figures_dir / "08_continuous_feature_decoding_r2.png",
)

plt.show()
else:
    print("No continuous regression table found.")

```





✓ CEBRA vs classical embeddings

```
# -----
# CEBRA versus classical latent embeddings
# -----
if not cebra_summary.empty:
    cebra_summary_sorted = (
        cebra_summary
        .sort_values("mean_r2_across_targets", ascending=False)
        .reset_index(drop=True)
    )

    save_table(
        cebra_summary_sorted,
        paths.tables_dir / "08_cebra_vs_classical_embedding_summary.csv",
    )

    display(cebra_summary_sorted)

fig, ax = plt.subplots(figsize=(7, 4))
ax.bar(
    cebra_summary_sorted["representation"],
    cebra_summary_sorted["mean_r2_across_targets"],
)
ax.axhline(0, linewidth=1)
ax.set_xlabel("Representation")
```

```

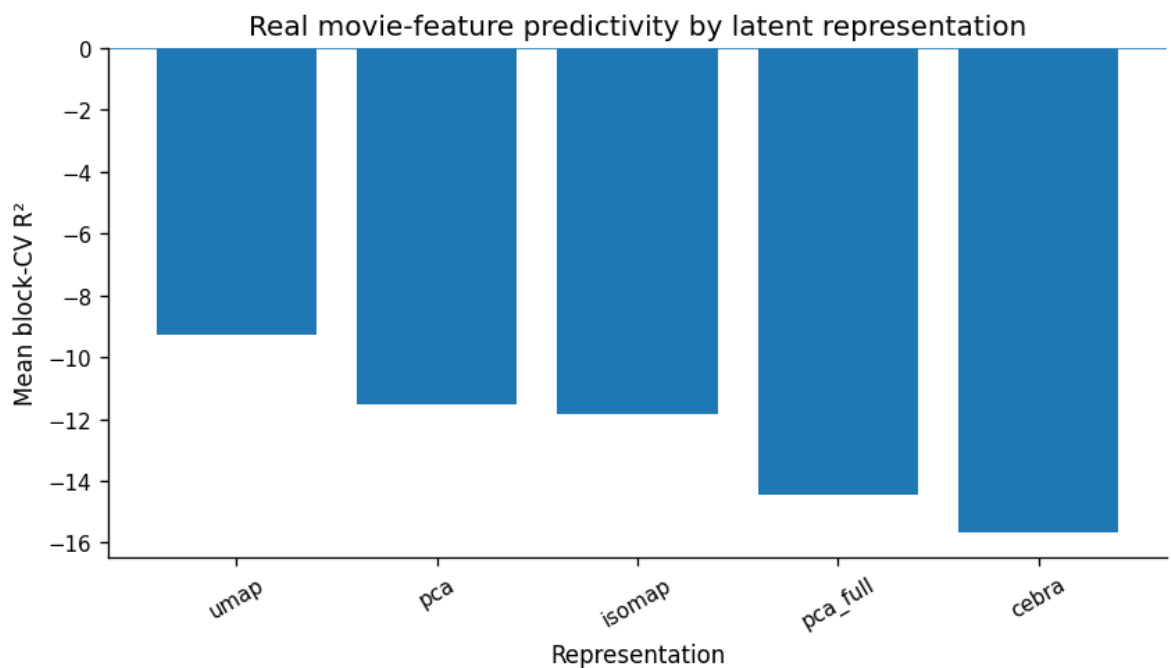
ax.set_ylabel("Mean block-CV  $R^2$ ")
ax.set_title("Real movie-feature predictivity by latent representation")
ax.tick_params(axis="x", rotation=30)

save_figure(
    fig,
    paths.figures_dir / "08_cebra_vs_classical_embedding_summary.png",
)

plt.show()
else:
    print("No CEBRA comparison summary found.")

```

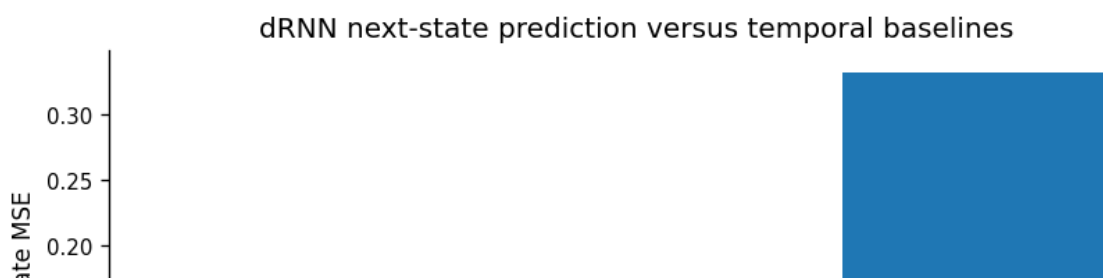
	representation	mean_r2_across_targets	mean_spearman_across_targets	n_targets
0	umap	-9.277270	0.093386	1000
1	pca	-11.514799	0.035455	1000
2	isomap	-11.837900	0.007714	1000
3	pca_full	-14.466427	0.102798	1000
4	cebra	-15.678328	-0.013523	1000

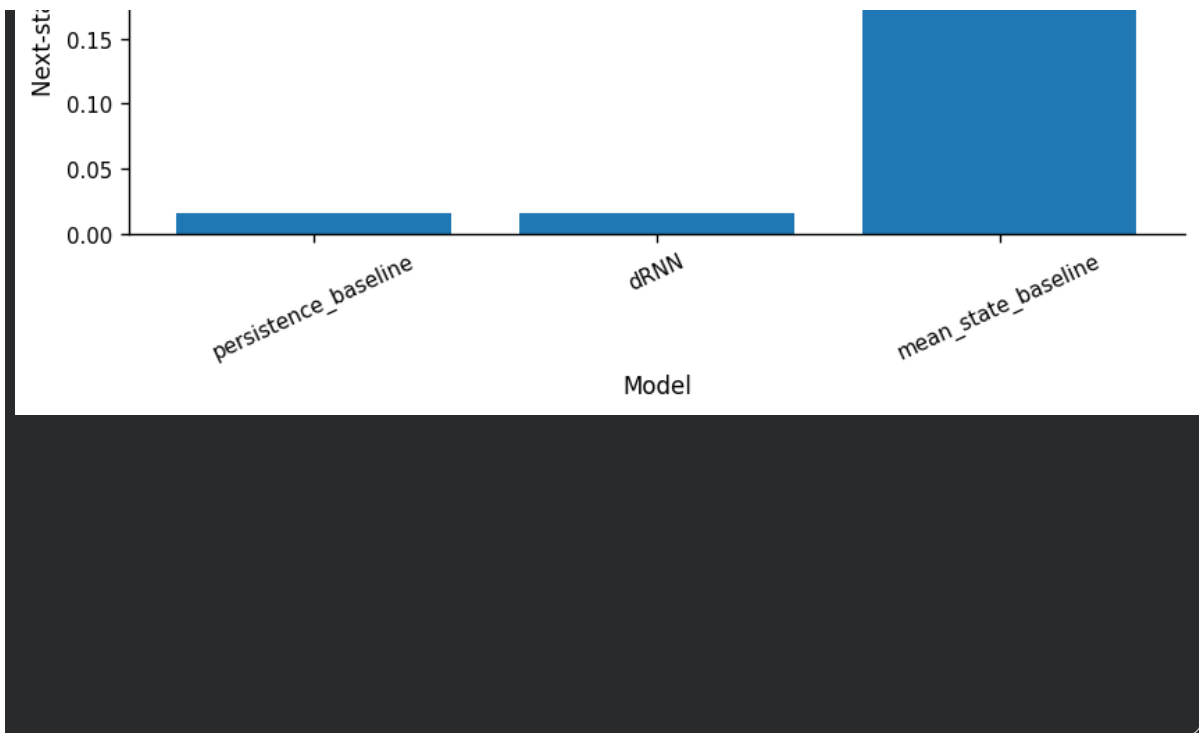


✓ dRNN baseline comparison

```
# -----  
# dRNN next-state prediction against temporal baselines  
# -----  
if not drnn_baseline.empty:  
    drnn_baseline_sorted = (  
        drnn_baseline  
        .sort_values("mse", ascending=True)  
        .reset_index(drop=True)  
    )  
  
    save_table(  
        drnn_baseline_sorted,  
        paths.tables_dir / "08_drnn_baseline_comparison_summary.csv",  
    )  
  
    display(drnn_baseline_sorted)  
  
    fig, ax = plt.subplots(figsize=(7, 4))  
    ax.bar(drnn_baseline_sorted["model"], drnn_baseline_sorted["mse"])  
    ax.set_xlabel("Model")  
    ax.set_ylabel("Next-state MSE")  
    ax.set_title("dRNN next-state prediction versus temporal baselines")  
    ax.tick_params(axis="x", rotation=25)  
  
    save_figure(  
        fig,  
        paths.figures_dir / "08_drnn_next_state_baseline_comparison.png",  
    )  
  
    plt.show()  
else:  
    print("No dRNN baseline table found.")
```

	model	n_samples	latent_dim	mse	rmse	mae	
0	persistence_baseline	870	3	0.015662	0.125146	0.082205	0.9
1	dRNN	870	3	0.015880	0.126015	0.085401	0.9
2	mean_state_baseline	870	3	0.333042	0.577098	0.501140	-0.0





```
# -----
# Geometry-decoding relationship using available reconstructed geometry metr
# -----
plot_df = integrated.dropna(
    subset=["mean_discrete_curvature", "balanced_accuracy"]
).copy()

# I keep only non-null models if the column exists.
if "is_null_baseline" in plot_df.columns:
    plot_df = plot_df[
        ~plot_df["is_null_baseline"].astype(str).str.lower().isin(["true", "
    ]).copy()

if len(plot_df):
    fig, ax = plt.subplots(figsize=(6.5, 4.5))

    ax.scatter(
        plot_df["mean_discrete_curvature"],
        plot_df["balanced_accuracy"],
        s=60,
        alpha=0.8,
    )

    for _, row in plot_df.iterrows():
        label = f"{row['embedding']}"
        ax.text(
            row["mean_discrete_curvature"],
            row["balanced_accuracy"],
            label,
            fontsize=8,
            -0.005, 0.005,
```

```

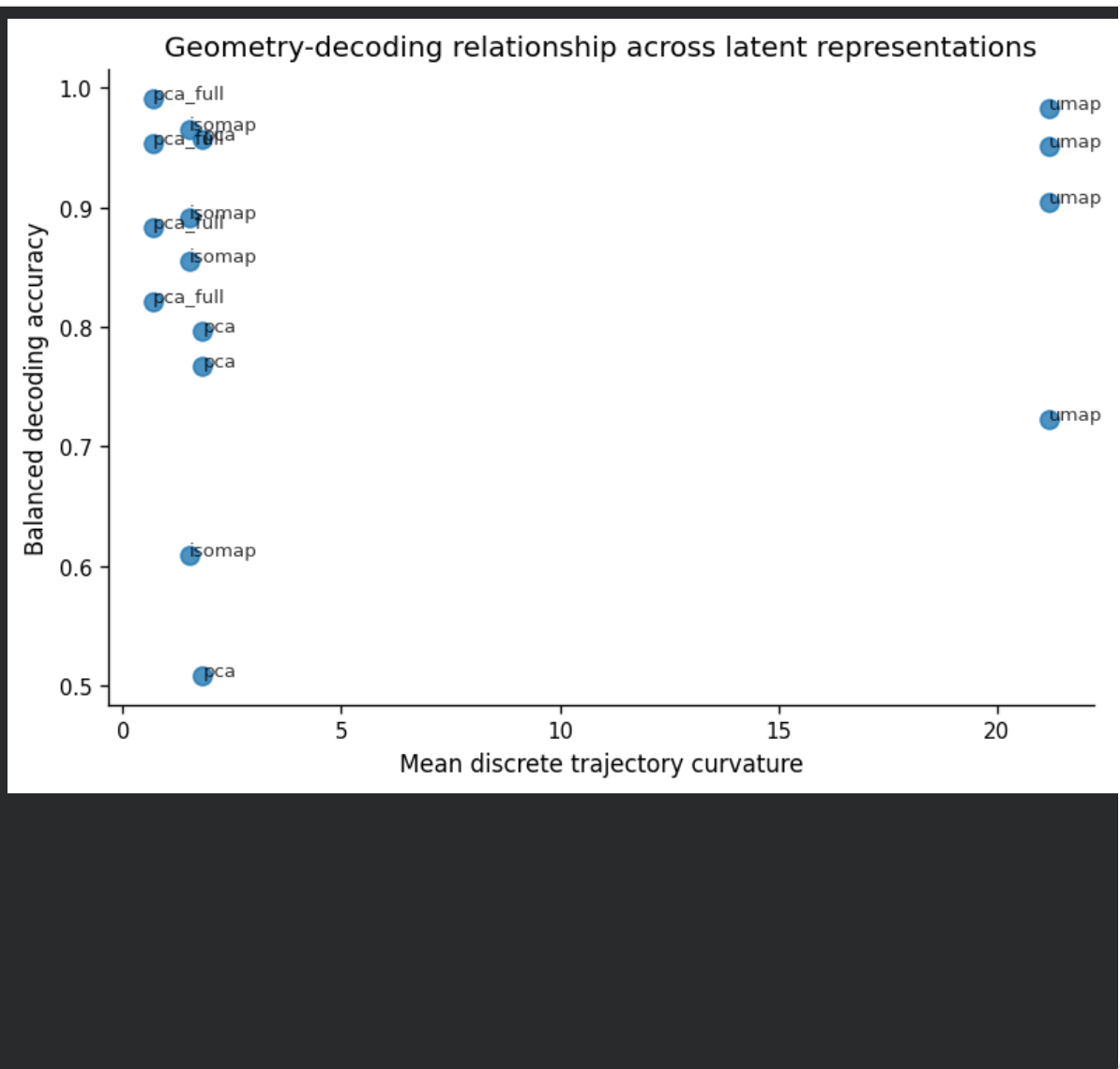
        alpha=0.8,
    )

    ax.set_xlabel("Mean discrete trajectory curvature")
    ax.set_ylabel("Balanced decoding accuracy")
    ax.set_title("Geometry-decoding relationship across latent representations")

    save_figure(
        fig,
        paths.figures_dir / "08_curvature_vs_decoding_accuracy.png",
    )

    plt.show()
else:
    print("No rows available for curvature-decoding plotting.")

```



```

# -----
# Best non-null decoding accuracy by embedding
# -----
plot_df = integrated.dropna(subset=["balanced_accuracy"]).copy()

```

```

if "is_null_baseline" in plot_df.columns:
    plot_df = plot_df[
        ~plot_df["is_null_baseline"].astype(str).str.lower().isin(["true", "
    ]).copy()

best_by_embedding = (
    plot_df
    .sort_values("balanced_accuracy", ascending=False)
    .groupby("embedding", as_index=False)
    .first()
    .sort_values("balanced_accuracy", ascending=False)
)

save_table(
    best_by_embedding,
    paths.tables_dir / "08_best_decoding_accuracy_by_embedding.csv",
)

if len(best_by_embedding):
    fig, ax = plt.subplots(figsize=(7, 4))

    ax.bar(
        best_by_embedding["embedding"],
        best_by_embedding["balanced_accuracy"],
    )

    ax.set_xlabel("Embedding")
    ax.set_ylabel("Best balanced accuracy")
    ax.set_title("Best real-feature decoding accuracy by embedding")
    ax.tick_params(axis="x", rotation=25)

    save_figure(
        fig,
        paths.figures_dir / "08_best_real_feature_decoding_by_embedding.png"
    )

    plt.show()
else:
    print("No decoding rows available for plotting.")

```

